PROGRAMACIÓN DE HORARIOS

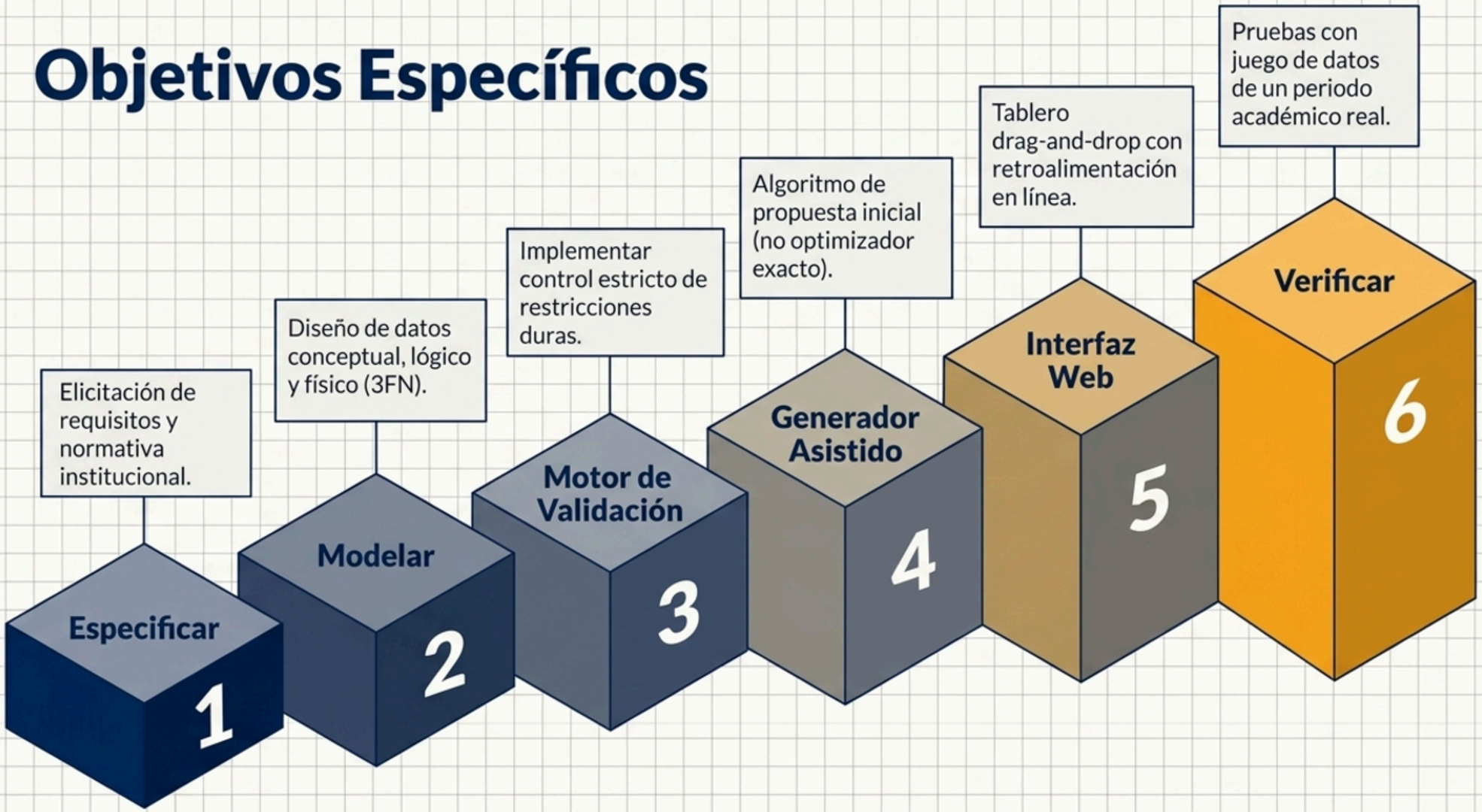
Presentado por KairósIngeniería S.A.S.

Hernando Javier García Mogollón, Jhonatan Lara Gómez, Juanita Mejía Pineda, Laura Gabriela Díaz Amaya y Santiago Pinzón Vásquez

Objetivo

Diseñar y construir un componente de software web que sistematice la elaboración del horario de clases de un periodo académico del programa de Ingeniería de Sistemas de la Universidad El Bosque, trasladando las reglas institucionales desde el conocimiento tácito del coordinador hacia una base de datos relacional parametrizable, de modo que el sistema genere una propuesta inicial de horario, valide automáticamente el cumplimiento de las restricciones duras y permita el ajuste manual asistido antes de la publicación.

Objetivos Específicos



Criterios de Éxito Medibles

100%

[E1] Flexibilidad

De parámetros (franjas, cupos, bloqueos) configurables sin tocar una sola línea de código fuente.

0

[E2] Fiabilidad

Horarios publicados con violaciones de restricciones duras. El sistema bloquea automáticamente la publicación.

≤ 25%

[E3] Eficiencia

Del tiempo actual invertido en el proceso de elaboración manual del horario de clases.

100%

[E4] Auditoría

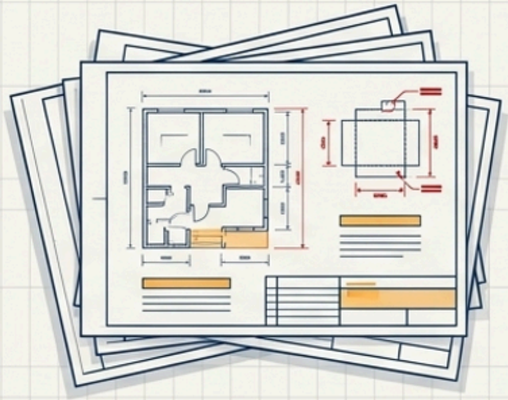
Trazabilidad de cambios sobre versiones publicadas (registro inmutable de usuario y fecha).

Alcance del Producto y Entregable

El Sistema (Producto)

- Módulo de parametrización (reglas como datos)
- Gestión del dominio (docentes, cursos, aulas)
- Motor de validación & Generador inicial
- Tablero drag-and-drop semanal
- Publicación y Exportación (CSV/PDF)
- Autenticación con roles y bitácora

La Documentación (Entregables)



- Documentos de requisitos (RF/RNF) y diseño (BD/Arquitectura)
- UI Wireframes y guía drag-and-drop
- Software funcional
- Plan de pruebas
- Manuales (Usuario y Técnico)
- Documento final IEEE y sustentación

Límites del Proyecto: Exclusiones, Supuestos y Restricciones

Fuera de Alcance (Exclusiones)

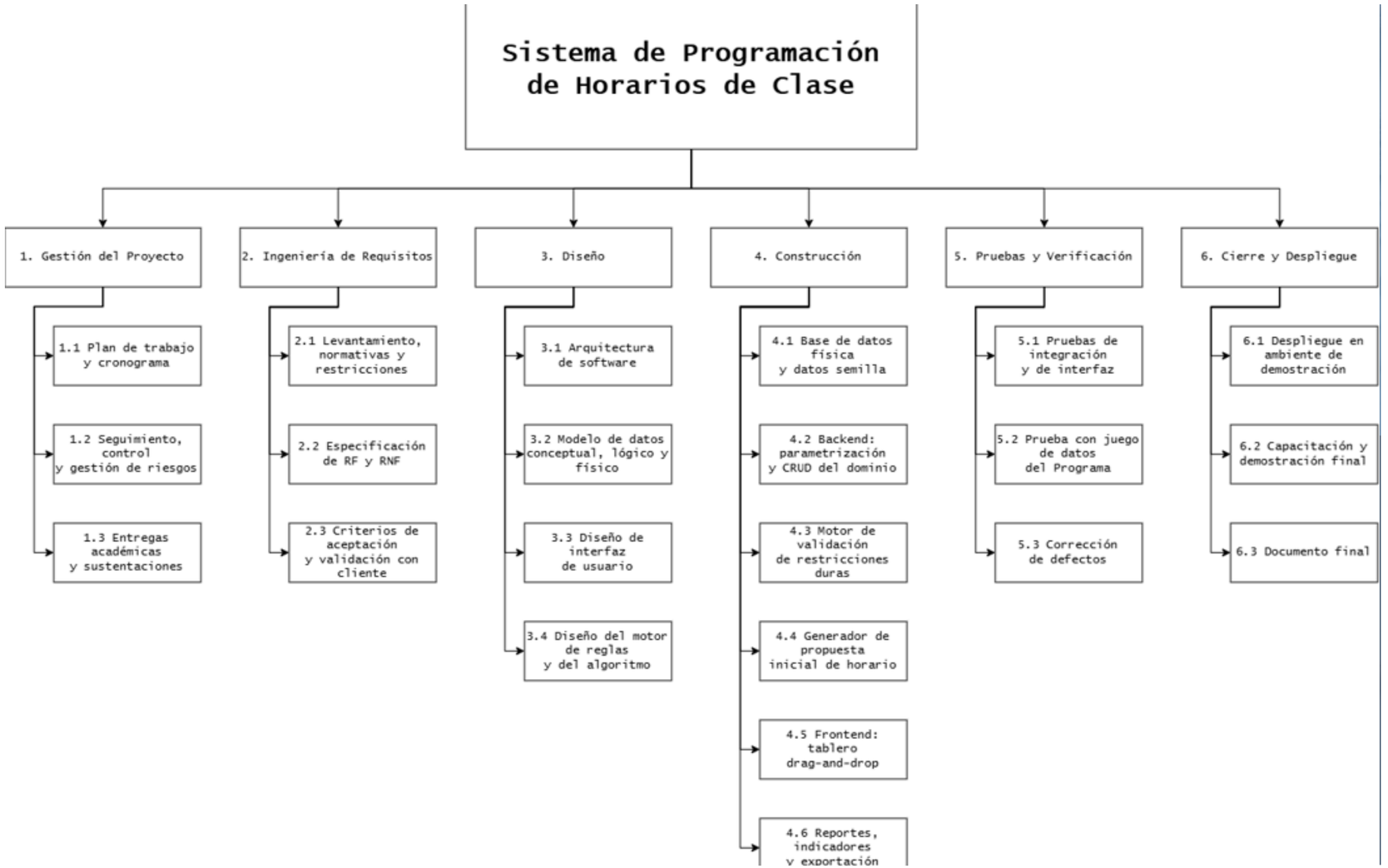
- Inscripción/matrícula de estudiantes.
- Integranción en linea con sistemas institucionales
- Programación de examenes o de otros programas.
- Cálculos de nómina y ejecución presupuestal.
- Reasignación automática de estudiante.
- App móvil o SSO institucional.
- Optimalidad matemática garantizada.

Supuestos

- Coordinador disponible para validar reglas.
- Normativas públicas vigentes.
- Datos suministrados en archivo plano.
- Bloques de clase base de 2 horas.

Restricciones

- Equipo de 5 estudiantes.
- Despliegue gratuito/local.
- Fecha de entrega fija atada al calendario.


Sistema de Programación de Horarios de Clase
1. Gestión del Proyecto
1.1 Plan de trabajo y cronograma
1.2 Seguimiento, control y gestión de riesgos
1.3 Entregas académicas y sustentaciones
2. Ingeniería de Requisitos
2.1 Levantamiento, normativas y restricciones
2.2 Especificación de RF y RNF
2.3 Criterios de aceptación y validación con cliente
3. Diseño
3.1 Arquitectura de software
3.2 Modelo de datos conceptual, lógico y físico
3.3 Diseño de interfaz de usuario
3.4 Diseño del motor de reglas y del algoritmo
4. Construcción
4.1 Base de datos física y datos semilla
4.2 Backend: parametrización y CRUD del dominio
4.3 Motor de validación de restricciones duras
4.4 Generador de propuesta inicial de horario
4.5 Frontend: tablero drag-and-drop
4.6 Reportes, indicadores y exportación
5. Pruebas y Verificación
5.1 Pruebas de integración y de interfaz
5.2 Prueba con juego de datos del Programa
5.3 Corrección de defectos
6. Cierre y Despliegue
6.1 Despliegue en ambiente de demostración
6.2 Capacitación y demostración final
6.3 Documento final

Requisitos Funcionales: Reglas y Dominio

A. Parametrización Institucional (RF-01, 02) — Reglas como Datos

Tiempo:
Configuración de franja hábil (L-V 7-22h, Sáb 7-13h), bloqueos globales (ej. 12-13h) y duración de bloques.

Capacidad:
Aforos máximos (40), tolerancia (10%), cupos mínimos (10) y sesiones máximas/semana (4).

B. Gestión del Dominio (RF-03 al 07) — Datos Estructurales

Docentes:
Tipos de vinculación, horas contratadas, restricciones de horario personal y cursos habilitados.

Infraestructura y Academia:
CRUD de Aulas (capacidad, equipos), Cursos (intensidad, requisitos de aula) y Grupos.

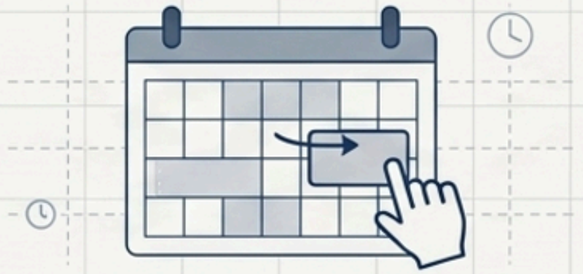
Requisitos Funcionales: El Corazón del Sistema

Generación y Ajuste (RF-08 a 11)

Las 8 Reglas Duras:

1. Cero cruces docentes.
2. Cero cruces aulas.
3. Respeta franja hábil.
4. Aula cumple requisitos.
5. Docente habilitado.
6. Disponibilidad respetada.
7. Límite horas contrato.
8. Límite sesiones/semana.

Reglas Blandas: Reporta (pero no bloquea) huecos y desbalance.



Publicación y Control (RF-12 a 15)

Control de Calidad:
Bloqueo absoluto de publicación si el validador detecta violación a reglas duras.

Salida y Auditoría:

- Versionamiento histórico.
- Exportación directa (CSV/PDF).
- Reportes estadísticos (aulas, carga).
- Autenticación segura y bitácora de auditoría inmutable.

Requisitos No Funcionales: Estándares de Calidad

Desempeño

Generar (60 grupos/30 docentes/20 aulas) en ≤ 60s. Validar UI en < 1s.

Usabilidad

Inducción ≤ 30 min. Errores descritos en lenguaje de dominio.

Mantenibilidad

Reglas duras aisladas en componente único (cambios sin tocar UI).

Modificabilidad

Algoritmo generador consumido vía interfaz; totalmente intercambiable.

Fiabilidad

Cero violaciones publicadas. Conservación de versiones previas.

Seguridad

Passwords con hash/sal. Transporte cifrado. Bitácora inmutable.

Compatibilidad

Chromium/Firefox (1366x768). Despliegue en ambiente local reproducible.

Normatividad

Cumplimiento estricto Ley 1581 (Hábeas Data) para datos de docentes.

Cronograma

Cronograma_Gantt_Proyecto_Horarios

Solución Técnica

1

El Validador

Recibe una asignación. Verifica instantáneamente las 8 reglas duras. Responde si es válida o no con motivo

2

El Generador

Algoritmo “voraz” (greedy). Ordena clases por dificultad. Busca la primera combinación valida. Si falla, la deja pendiente para no trabar el proceso.

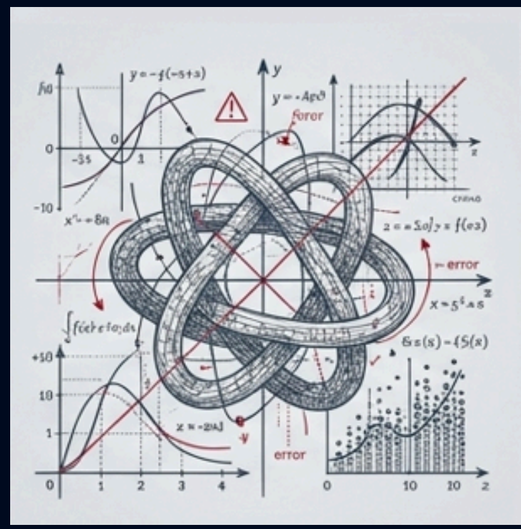
3

El Ajuste

El coordinador afina la propuesta arrastrando bloques. El validador vigila cada movimiento en tiempo real

Justificación Técnica

Solución Exacta



- **Enfoque:** Solucionadores matemáticos completos.
- **Problema:** Requiere altísimo tiempo de desarrollo, herramientas de terceros y es difícil de defender bajo las restricciones del proyecto.

Nuestra Solución

- **Simpleza**: Construible, explicable y mantenible.
- **Pragmatismo**: Un horario válido y seguro (no matemáticamente perfecto) es suficiente para el afinamiento humano.
- **Extensibilidad**: Algoritmo aislado tras una interfaz (RNF-04). Permite conectar un optimizador complejo en el futuro sin reescribir el sistema.

GRACIAS